CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

DESIGN, ANALYSE AND VALIDATE A CLOUD-BASED SOLUTION FOR MIGRATING A LEGACY UNIVERSITY MANAGEMENT SYSTEM

Smart Campus Cloud Migration

Student Information	Details
Student Name	DANIEL SAM K.
Register Number	192524081
Department / Section	B TECH AI & DS
Academic Year / Semester	2ND YEAR
Faculty Name	DR.R.LATHA
Submission Date	31-10-26

Total Marks: 50 | Duration: 1 Week

DECLARATION AND SCOPE

This report addresses all eleven tasks in the supplied CSA15 CO1 assignment brief. It uses the university scenario stated in the question: 25,000 students, 2,000 staff, four campuses, twelve ageing physical servers, low average utilisation, seasonal peaks, monolithic applications and no disaster-recovery site.

The proposed design is an academically reasoned reference architecture. Numerical costs are planning estimates based on explicit assumptions, not vendor quotations. Task 9 includes reproducible proof-of-concept instructions and sample outputs; these are not represented as proof that a cloud account, VM, container or hosted service has actually been executed. Authentic screenshots and real execution logs must be added by the student after running the steps.

EXECUTIVE SUMMARY

The university's current infrastructure is inefficient because twelve dedicated servers average only about 15% CPU utilisation while still reaching more than 95% during admissions and examination periods. The current design also has long procurement lead times, tightly coupled monolithic applications, no standard APIs and no disaster-recovery site. A cloud migration can address these weaknesses through pooled infrastructure, virtualization, managed platforms, elastic capacity, API-based integration and geographically separate recovery.

The recommended baseline is a hybrid-by-design architecture using a public-cloud landing zone for elastic web and application services, managed databases and object storage, while retaining selected legacy integrations on-premises during transition. The five applications should not all be treated identically: SIS and selected legacy functions may initially use IaaS VMs; modernized SIS, LMS integrations and library services should move towards PaaS/container platforms; email should normally be SaaS; and the exam portal should use isolated, autoscaled application capacity with strong identity and monitoring controls.

For the Day-4 challenge, the original design should be MODIFIED rather than simply retained. A 10× peak-load requirement, mobile access, 40 external colleges and 99.9% availability require an API gateway, tenant-aware authorization, multi-zone deployment, queue-based asynchronous processing, stronger observability, tested disaster recovery and capacity/load testing. A multi-region active-passive recovery design is recommended where the budget does not support full active-active operation.

TASK 1 – INTRODUCE CLOUD COMPUTING AND MAP IT TO THE SCENARIO

1.1 Definition of cloud computing

Cloud computing is a model for providing convenient, on-demand network access to a shared pool of configurable computing resources—such as servers, storage, networks, applications and services—that can be rapidly provisioned and released with limited management effort or service-provider interaction. Instead of purchasing and maintaining every server, the university consumes infrastructure and platform capabilities as services and pays according to agreed capacity or usage.

Cloud computing is not simply moving a physical server to another building. It changes how resources are provisioned, pooled, secured, monitored, scaled and paid for. The university can request additional application instances during admissions, reduce them after the peak, and use managed services for databases, backups and identity without owning all underlying hardware.

1.2 Five essential characteristics and scenario mapping

Characteristic	Meaning	Application to the university	Problem addressed
On-demand self-service	A consumer can provision computing resources when required without waiting for manual provider action.	Authorized IT staff provision a test environment, database or application instance through a portal or infrastructure-as-code pipeline.	Reduces the 6–8 week procurement delay for new servers.
Broad network access	Services are available over standard networks and accessed through common clients such as browsers, laptops and phones.	Students, faculty and administrators access SIS, LMS, library and exam services across four campuses and remotely.	Removes dependence on a single campus network or locally installed client.
Resource pooling	Provider resources serve multiple consumers using a multi-tenant model, with logical isolation and dynamic allocation.	Compute, storage and network capacity are pooled; the university receives isolated resources and can separate affiliated colleges by tenant.	Reduces idle dedicated capacity and improves utilisation.
Rapid elasticity	Resources can scale out/in or up/down quickly as demand changes.	Add application replicas and database read capacity during admissions and result publication; scale down after peaks.	Addresses seasonal peaks above 95% CPU without permanently buying peak-sized servers.
Measured service	Usage is metered, monitored and reported so consumption can be controlled and billed.	Track compute hours, storage, database I/O, bandwidth and backup usage by application or department.	Improves cost visibility and supports chargeback/showback.

1.3 Cloud deployment models

Model	Description	Advantages	Limitations	Suitability
Public cloud	Infrastructure is owned and operated by a cloud provider and delivered to many customers over the internet.	Fast provisioning, broad services, elasticity and no university-owned data-centre hardware.	Shared-responsibility security, internet dependency, recurring usage cost and possible data-residency constraints.	Strong fit for public-facing portals, development/test, elastic application tiers and managed services.
Private cloud	Cloud environment dedicated to one organisation, on-premises or hosted.	Greater control over configuration, location and specialised compliance needs.	Higher cost and operational responsibility; university must operate or pay for dedicated capacity.	Useful only for workloads with justified strict control or legacy constraints.
Hybrid cloud	Combination of private/on-premises resources and public-cloud services connected through secure networking and identity.	Allows phased migration, keeps selected legacy systems temporarily in place and uses public cloud for elasticity.	Integration, identity, monitoring and networking become more complex.	Best transition model for the university while legacy systems are modernised.
Community cloud	Infrastructure shared by organisations with common requirements, such as a group of educational institutions.	Potentially shared cost and common governance or sector-specific controls.	Requires agreement on ownership, governance, access and cost allocation.	Could be considered for a consortium of affiliated colleges, but it is not required for the initial migration.

Recommendation: adopt a hybrid migration strategy with public-cloud services as the target for elastic and managed workloads. A private-cloud deployment for the entire university is not justified by the stated facts because it would preserve much of the capital and operational burden. A

community model may be explored later if the 40 affiliated colleges agree on common governance and funding.

1.4 Three major on-premises problems and cloud response

1. Low utilisation and idle capacity: twelve servers average approximately 15% CPU utilisation. Virtualization and pooled cloud resources allow several workloads to share fewer physical hosts, while autoscaling adds capacity only when demand rises.
2. Peak-load failures and slow procurement: CPU peaks exceed 95%, but buying a new physical server takes 6–8 weeks. Elastic compute, load balancing and automated scaling can add instances within minutes, subject to configured quotas and application readiness.
3. No disaster recovery and monolithic integration: a single data centre creates a major failure domain, while monolithic applications lack reusable interfaces. Multi-zone deployment, backup replication, tested recovery and API-based service boundaries reduce these weaknesses.

1.5 Benefits and limitations

Benefits include improved elasticity, faster provisioning, better average resource utilisation, reduced upfront capital expenditure, access to managed databases and security tooling, remote access, improved backup options and easier integration through APIs. Cloud services can also support experimentation because test environments can be created and removed quickly.

Limitations include dependence on internet connectivity and provider availability, recurring charges that can grow unexpectedly, vendor lock-in, data-protection and residency obligations, migration complexity, skills requirements, and the possibility that a poorly designed application will not scale simply because it is hosted in the cloud. Cloud adoption does not automatically make an insecure or unreliable application secure or reliable.

Therefore, the university should use identity controls, encryption, budget alerts, tagging, cost reviews, backup tests, portability standards and a phased migration. It should also test the application under realistic peak load before promising service levels.

TASK 2 – TRACE THE EVOLUTION AND HARDWARE EVOLUTION

2.1 Timeline of computing evolution

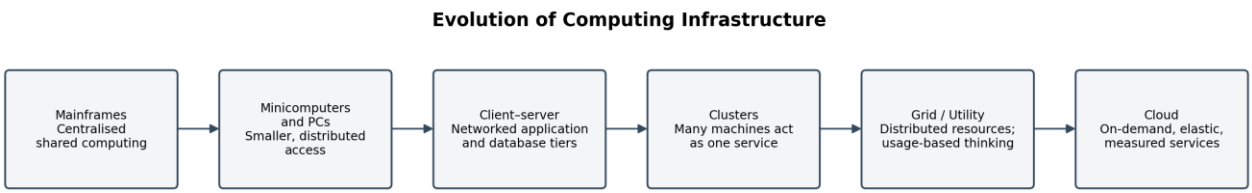


Figure 1. Simplified evolution from centralised computing to cloud services.

Stage	Main idea	Limitation addressed / contribution

Mainframes	Large central computers served many users through terminals.	Centralised processing and shared access replaced separate computing equipment for every user, but systems were expensive and centrally controlled.
Minicomputers	Smaller, less expensive multi-user systems.	Made departmental computing more accessible than a single large mainframe.
Personal computers	Computing power moved to individual desks.	Improved user autonomy, but data and applications became distributed and harder to manage consistently.
Client–server	Clients request services from central application/database servers.	Enabled shared data and network applications, but servers could become bottlenecks and single points of failure.
Clusters	Multiple connected servers work together as a service.	Improved performance and availability by distributing work across machines.
Grid computing	Distributed resources from different locations are coordinated for large jobs.	Enabled resource sharing across administrative boundaries, although management and scheduling could be complex.
Utility computing	Computing is treated as a metered utility.	Introduced consumption-based charging and the idea of paying for capacity used.
Cloud computing	Virtualized, pooled resources and managed services are provisioned over networks on demand.	Combines resource pooling, automation, elasticity, measured usage and service-level management.

2.2 Hardware evolution that enabled cloud computing

- Multi-core processors: several processing cores in one chip allow a server to run multiple workloads concurrently. Virtual machines and containers can schedule workloads across available cores.
- Commodity x86 servers: large fleets of relatively standard servers can be assembled into scalable data-centre capacity. Standardisation reduces the need for a single expensive specialised machine.
- High-capacity storage: lower-cost disks, flash storage, storage arrays and distributed object stores support large volumes of student records, documents, logs and backups.
- High-speed networking: faster Ethernet, fibre and software-defined networking connect compute and storage resources with lower latency and higher throughput.
- Data-centre engineering: redundant power, cooling, physical security, network paths and automated operations make it practical to operate large shared infrastructure.
- Hardware-assisted virtualization: processor features such as Intel VT-x and AMD-V help hypervisors run guest operating systems efficiently with isolation.

2.3 How each stage solved the previous limitation

The progression is not a simple replacement sequence; many models continue to coexist. Mainframes centralised expensive processing. Minicomputers lowered the cost of departmental access. PCs moved computing closer to individual users but created management and data-sharing problems. Client–server systems re-centralised shared data and services. Clusters improved scale and resilience beyond a single server. Grid computing coordinated geographically distributed resources. Utility computing introduced metering and consumption-based economics. Cloud computing combined these ideas with virtualization, APIs, automation and rapid provisioning.

2.4 Where the university's current system fits

The university's present environment most closely resembles a traditional dedicated-server client–server data centre. Each application has its own physical server, which indicates limited pooling and

consolidation. It has some networked application behaviour, but the scenario does not describe elastic clusters, automated provisioning, managed services or a recovery region. Its 15% average CPU use and peaks above 95% illustrate the weakness of fixed-capacity provisioning: the university pays for resources that sit idle for much of the year yet still struggles during short high-demand periods.

TASK 3 – ANALYSE INTERNET SOFTWARE EVOLUTION

3.1 Evolution of internet software

Stage	Characteristics	Relevance to university
Static web pages	Mostly fixed HTML pages; content changes only when files are updated by a publisher.	Useful for public notices and static information, but insufficient for interactive student records.
Dynamic web	Server-side programs generate pages using databases and user input.	Supports login, student profiles, course registration and result display.
Web 2.0	Interactive, participatory applications with asynchronous updates and richer user experience.	Supports portals, discussion, notifications, collaborative learning and responsive interfaces.
Service-Oriented Architecture (SOA)	Business functions are exposed as reusable, loosely coupled services with defined contracts.	Allows SIS, library, LMS and payment functions to be integrated without sharing one monolithic codebase.
Mashups	Applications combine data or services from multiple sources through APIs.	A student portal can combine timetable, library status, fee balance and campus notices.
Cloud-hosted applications	Applications run on provider infrastructure or managed platforms and scale through cloud capabilities.	Enables mobile access, elastic capacity, managed databases and deployment across availability zones.

3.2 Role of browsers, HTTP, AJAX and APIs

- Browsers provide a common client platform. Students can use a modern browser without installing a separate desktop application on every device.
- HTTP/HTTPS carries requests and responses between clients and web services. HTTPS adds transport encryption and server authentication through TLS.
- AJAX (Asynchronous JavaScript and XML) is a historical name for browser techniques that send background requests and update parts of a page without a full reload. Modern implementations commonly use JSON with fetch or similar APIs.
- APIs define how software components communicate. They specify endpoints, methods, data formats, authentication and error behaviour. They allow a mobile app or partner college to consume selected university functions without direct database access.

3.3 Desktop, client–server and web-delivered software

Dimension	Desktop software	Client–server software	Web-delivered software
Installation	Installed on each client device.	Client software may need installation or configuration.	Usually accessed through a browser or mobile client.
Updates	Often updated on each device.	Client and server updates may both be required.	Server-side updates are centrally deployed; browser clients are generally simpler.
Data access	May store data locally or connect to servers.	Central server stores shared data.	Application services and databases are hosted centrally or in cloud services.
Scaling	Local device capacity limits individual use.	Server capacity must be planned and scaled.	Can use load balancing, replicas, managed services and autoscaling.

Integration	Often depends on file exchange or custom connectors.	Can integrate through defined network protocols.	APIs and web standards make integration more straightforward.
University fit	Useful for specialised offline or laboratory tools.	Describes many legacy campus systems.	Preferred access model for student, staff and partner-facing services.

3.4 Impact on university application design

The university should gradually decompose monolithic functions into service boundaries based on business capability. For example, student identity, course enrolment, result publication, fee payment and library search can become separately versioned services. A web or mobile client should communicate through an API gateway rather than connect directly to databases. The design should use documented JSON schemas, standard authentication, input validation, pagination, rate limiting and consistent error responses.

The transition should be incremental. A strangler pattern can place an API facade in front of the legacy system and move one function at a time to a new service. This reduces the risk of a single large rewrite and allows legacy and modern components to coexist during migration.

TASK 4 – APPLY SERVER VIRTUALIZATION

4.1 How a hypervisor works

A hypervisor, also called a virtual machine monitor, abstracts physical CPU, memory, storage and network devices so that multiple virtual machines can run on one physical host. It schedules virtual CPUs onto physical CPU cores, allocates guest memory, presents virtual disks and network adapters, and enforces isolation between guest environments. Each VM can run its own guest operating system.

4.2 Type-1 and Type-2 hypervisors

Type	Operation	Examples	Advantages / limitations
Type-1 (bare-metal)	Runs directly on physical server hardware; guest VMs run above it.	VMware ESXi, Microsoft Hyper-V, Xen, KVM-based platforms.	Generally preferred for production server consolidation because it avoids a general-purpose host OS layer. Requires appropriate administration and hardware compatibility.
Type-2 (hosted)	Runs as an application on a conventional host operating system.	Oracle VirtualBox, VMware Workstation.	Convenient for labs, learning and desktop testing. Adds a host OS layer and is not normally the first choice for a large production server farm.

4.3 Virtualization techniques

Technique	How it works	Strengths	Trade-offs / use
Full virtualization	Hypervisor presents virtual hardware so an unmodified guest OS can run.	Broad OS compatibility and strong VM isolation.	More overhead than a lightweight container; useful for legacy SIS or applications needing a separate OS.
Para-virtualization	Guest OS is aware of virtualization and uses hypercalls or specialised drivers for efficient interaction.	Can reduce overhead for supported guests and I/O paths.	Requires guest or driver support; less transparent than full virtualization.

OS-level virtualization (containers)	Processes share the host kernel but are isolated using namespaces, control groups and container runtime features.	Fast startup, high density and consistent packaging.	Not a separate kernel per container; kernel compatibility and isolation need careful management. Suitable for modern API services.
Hardware-assisted virtualization	CPU/chipset features assist hypervisor execution and memory management.	Improves performance and supports efficient guest isolation.	It is an enabling mechanism used with hypervisors, not a separate workload packaging model.

4.4 Proposed VM and container placement

Workload	Initial placement	Reason
Legacy SIS	VM initially; modernise selected functions into containers/PaaS later.	Legacy OS and application dependencies may require a complete guest environment. Migration can be staged.
LMS	Managed SaaS if a suitable institutional product meets requirements; otherwise containers/PaaS.	Learning platforms need web scaling and integrations. SaaS reduces patching if functionality and data controls meet requirements.
Library system	VM for legacy integrated library software; containers/PaaS for new search and discovery APIs.	Preserves existing dependencies while allowing public-facing search services to scale independently.
Exam portal	Containers on a managed orchestration platform or autoscaled VM group, with isolated exam workers.	Rapid scaling, repeatable deployment and separation of exam workloads from other applications.
Email	SaaS.	Email is a commodity collaboration service; operating mail servers adds patching, spam filtering, availability and storage burdens.

4.5 Consolidation calculation

Given: 12 physical servers; average CPU utilisation per server = 15%; target average utilisation per consolidated host = 70%. The calculation below is a first-order CPU-capacity estimate. It assumes comparable CPU capacity across the twelve servers and that their average workloads can be pooled. It does not prove that memory, storage IOPS, network throughput or peak demand will fit.

Total current average CPU load = 12 servers × 15% = 180% of one-server capacity\n\nRequired hosts at 70% target utilisation = 180% ÷ 70%\n = 2.571 hosts\n\nRound up to the next whole host = 3 physical hosts (theoretical minimum)

Interpretation: three equally capable hosts would carry the measured average CPU load at approximately 60% utilisation each (180% ÷ 3 = 60%), below the 70% planning target. However, three hosts provide limited failure tolerance. If one host fails, the remaining two would need to carry 180% total load, or about 90% each, before accounting for peak demand. Therefore, a practical production design should use at least four hosts for N+1 resilience in a private virtualisation cluster, subject to benchmark validation. For cloud migration, the equivalent capacity should be sized using instance metrics and autoscaling rather than treating this estimate as a fixed cloud host count.

4.6 Peak-load caution and capacity validation

The average utilisation calculation can hide severe peaks. The assignment states peaks above 95%, so the university must collect time-series CPU, memory, disk latency, database connections and network metrics at short intervals across admissions and exam periods. A safe design should retain headroom, test failover, and avoid assuming that all twelve servers peak at the same time or that their peaks are perfectly independent. If correlated peaks occur, the required capacity may be substantially higher.

4.7 Benefits and risks

- Live migration can move a running VM between compatible hosts with limited interruption, supporting maintenance and load balancing. It depends on shared or replicated storage, compatible processors and sufficient network bandwidth.
- Snapshots capture a point-in-time VM disk/state for short-term rollback or testing. A snapshot is not a substitute for an independent, application-consistent backup.
- Resource over-commitment allows allocated virtual CPU or memory to exceed physical capacity based on expected usage. Excessive over-commitment can cause contention, swapping and unpredictable latency.
- Noisy-neighbour risk occurs when one workload consumes shared CPU, memory bandwidth, storage IOPS or network capacity and degrades another workload.
- Hypervisor failure can affect many VMs on one host. Clustering, redundant hosts, patch management, monitoring and tested recovery reduce the blast radius.
- Security isolation can fail through misconfiguration, vulnerable hypervisors, exposed management interfaces or excessive privileges. Use hardened hosts, role-based access, MFA, patching and network segmentation.

TASK 5 – APPLY WEB SERVICES

5.1 Web services and their role in cloud/SOA

A web service exposes a software function through a network-accessible interface using standard protocols and data formats. In a Service-Oriented Architecture, a business capability is made available as a reusable service with a defined contract. Cloud applications use web services to connect browser clients, mobile applications, databases, payment providers and external institutions.

5.2 SOAP, WSDL, UDDI and REST

Technology	Meaning / role	Typical format or mechanism	When useful
SOAP	Protocol for structured message exchange with defined envelope and processing rules.	XML envelope; often used with WS-* standards.	Useful where formal contracts, enterprise messaging standards or established SOAP integrations are required.
WSDL	Web Services Description Language; describes SOAP service operations, messages and bindings.	XML service description.	Allows clients and tools to understand and generate calls to a SOAP service.
UDDI	Registry model for publishing and discovering web services.	Service metadata and registry entries.	Historically used for service discovery; many modern systems use API catalogues instead.
REST	Architectural style that uses resource-oriented URIs and standard HTTP semantics.	Often JSON over HTTPS; can also use XML or other representations.	Well suited to mobile apps, browser clients and partner integrations due to simplicity and broad tooling.

REST is not a protocol equivalent to SOAP; it is an architectural style that commonly uses HTTP. WSDL is a description language associated with web-service contracts, especially SOAP services. UDDI is a registry concept rather than a message format.

5.3 Three university web services

The following endpoint designs are illustrative. They should be published behind an API gateway, use HTTPS, authenticate callers, validate inputs and avoid exposing direct database access.

Service A – Student result lookup (REST)

Purpose: allow an authenticated student to retrieve only their own published result, or allow an authorised college administrator to retrieve records within that college's permitted tenant scope.

```
GET /api/v1/students/{studentId}/results?semester=2026- odd\nAuthorization: Bearer <access_token>\nAccept: application/json\n\nResponse: HTTP 200 OK\n{\n  "studentId": "STU10245",\n  "semester": "2026-odd",\n  "results": [\n {"courseCode": "CSA15", "grade": "A", "status": "PUBLISHED"}\n  ]\n}
```

REST is appropriate because the operation retrieves a resource, uses HTTP GET semantics and returns compact JSON suitable for a mobile application. The service must verify the caller's identity and student/tenant permissions; a student ID alone is not authorisation.

Service B – Fee payment initiation (REST)

```
POST /api/v1/payments\nAuthorization: Bearer <access_token>\nContent-Type: application/json\nIdempotency-Key: 4f3d...\n\nRequest:\n{\n  "studentId": "STU10245",\n  "invoiceId": "INV7781",\n  "amount": 12500.00,\n  "currency": "INR",\n  "returnUrl": "https://portal.example.edu/payment-result"\n}\n\nResponse: HTTP 202 Accepted\n{\n  "paymentId": "PAY90021",\n  "status": "PENDING",\n  "redirectUrl": "https://payment-provider.example/checkout"\n}
```

REST is suitable for the mobile and web front ends. Payment processing must use a trusted payment gateway; the university should not store raw card details. The idempotency key prevents accidental duplicate charges when a client retries after a timeout. The final payment status should be confirmed through a signed provider callback or server-to-server verification, not merely by trusting a browser redirect.

Service C – Library catalogue search (REST)

```
GET /api/v1/library/search?q=distributed%20systems&page=1&pageSize=10\nAuthorization: Bearer <access_token>\nAccept: application/json\n\nResponse: HTTP 200 OK\n{\n  "query": "distributed systems",\n  "page": 1,\n  "pageSize": 10,\n  "total": 2,\n  "items": [\n    {"itemId": "LIB110", "title": "Distributed Systems", "availability": "AVAILABLE"},\n    {"itemId": "LIB245", "title": "Cloud Computing", "availability": "ON_LOAN"}\n  ]\n}
```

REST is appropriate because catalogue search is read-oriented, naturally maps to a resource query and benefits from caching with carefully selected cache-control rules. Personal borrowing records must not be returned by a public catalogue endpoint.

5.4 When SOAP may still be justified

SOAP may be retained for an existing finance, government or enterprise integration that requires a formal WSDL contract, XML signatures, WS-Security or a provider-mandated SOAP interface. The university should not rewrite a stable SOAP integration solely to use REST. A gateway or adapter can expose a simpler REST interface to mobile clients while translating to the legacy SOAP service internally.

5.5 API governance and security

- Use versioned endpoints such as /api/v1 and publish a machine-readable OpenAPI specification for REST services.

- Use OAuth 2.0 / OpenID Connect, short-lived access tokens, MFA for privileged users and server-side authorization checks.
- Apply rate limits and quotas per client and tenant; use request-size limits, schema validation and input sanitisation.
- Use correlation IDs, structured logs, audit events and metrics without recording passwords, tokens or unnecessary sensitive student data.
- Use idempotency for payment and enrolment operations; use queues and retry policies for asynchronous tasks.
- Maintain backward compatibility and a documented deprecation period for partner colleges.

TASK 6 – ANALYSE THE SERVICE MODELS: IAAS, PAAS AND SAAS

6.1 Definitions and responsibility layers

Infrastructure as a Service (IaaS) supplies fundamental compute, storage and networking resources. The customer manages the operating system, middleware, runtime, applications and much of the configuration, while the provider manages the physical data centre and underlying hardware.

Platform as a Service (PaaS) supplies a managed application platform or runtime. The provider manages the infrastructure and much of the operating system/runtime platform; the customer focuses on application code, data, configuration and identity/access policies.

Software as a Service (SaaS) supplies a complete application to end users. The provider operates the application and underlying stack; the customer manages users, access, configuration, data governance and appropriate retention/export settings.

Layer	On-premises	IaaS customer	PaaS customer	SaaS customer
Physical data centre / hardware	University	Provider	Provider	Provider
Virtualization / base infrastructure	University	Provider	Provider	Provider
Operating system	University	Customer	Provider (managed)	Provider
Middleware / runtime	University	Customer	Provider (managed platform)	Provider
Application code / software	University	Customer	Customer	Provider
Data, users and access configuration	University	Customer	Customer	Shared responsibility; customer governs use and access

The boundary is not identical for every vendor product. The university must check the provider's shared-responsibility documentation and contract. Even with SaaS, the institution remains responsible for appropriate user provisioning, data classification, lawful use, access reviews and retention decisions.

6.2 Mapping the five university applications

Application	Recommended model	Reasoning	Key control

Student Information System (SIS)	IaaS initially, then selective PaaS/container modernisation.	SIS is mission-critical and likely contains legacy dependencies and complex integrations. A VM-based first migration can reduce immediate code changes; new functions should be extracted into managed services over time.	Strong IAM, encryption, database resilience, audit trails and tested backups.
Learning Management System (LMS)	SaaS preferred if requirements, integration and data terms are acceptable; otherwise PaaS.	A mature LMS is a standard institutional capability. SaaS reduces infrastructure and patching work; PaaS is appropriate if deep customisation or control is required.	SSO, data export, accessibility, integration and service-level review.
Library System	IaaS for legacy integrated library system; PaaS for new search/API components.	Existing circulation/catalogue software may have vendor-specific dependencies. A staged approach preserves continuity while allowing search and discovery services to scale.	Data integrity, catalogue backups, identity integration and vendor support.
Exam Portal	PaaS/container platform with isolated worker pools; IaaS fallback for legacy components.	Exam demand is highly bursty and requires repeatable deployment, scaling and separation from ordinary portal traffic.	Capacity tests, immutable exam logs, anti-cheating controls, time synchronisation and recovery.
Email	SaaS.	Email, calendaring, spam filtering and collaboration are standard services with substantial operational overhead if self-hosted.	MFA, domain security, retention, e-discovery, data location and account lifecycle controls.

6.3 Detailed reasoning

The SIS should not be forced into SaaS merely because SaaS reduces maintenance. Student records are central to university operations and may have specialised workflows and integrations. An IaaS landing zone can provide a controlled transition while the university documents interfaces, removes hard-coded dependencies and modernises selected services. Over time, stable business functions can move to PaaS or managed services.

For the LMS and email systems, the university should evaluate mature SaaS offerings because the provider can manage routine patching, upgrades and much of the availability engineering. The decision must still examine data processing terms, integration, identity federation, accessibility, data export and exit costs. A SaaS product that cannot meet institutional teaching or compliance needs is not automatically the correct choice.

The exam portal needs a different design from the SIS. Exam periods create short but intense bursts, and failures can affect fairness and student outcomes. A container platform can scale stateless web/API replicas while queue-based workers process notifications and background tasks. Exam sessions should be isolated by course or cohort where appropriate, and the database must be tested for write contention.

6.4 Comparison of service models

Dimension	IaaS	PaaS	SaaS
Control	Highest control over OS, middleware and application configuration.	Control over code, data and platform configuration; less control over runtime internals.	Lowest technical control; configuration and access policies remain.
Cost pattern	Compute/storage/network consumption plus staff effort and OS licensing.	Usage-based platform charges; less infrastructure administration.	Subscription or per-user pricing; add-on and storage charges may apply.
Scalability	Possible, but customer designs and operates scaling mechanisms.	Often provides built-in scaling and managed runtime capabilities.	Provider scales the product; customer may have plan, quota or tenant limits.

Maintenance effort	High: OS patching, runtime, backups and application operations.	Medium: code and data remain customer's responsibility; platform maintenance is reduced.	Lower infrastructure effort; customer still manages configuration, users and governance.
Vendor lock-in	Can be lower with portable OS images, but proprietary networking and managed add-ons can create lock-in.	Can be higher if application relies on proprietary runtimes, databases or event services.	Potentially high due to proprietary workflows, data formats and migration/export constraints.

6.5 Provider examples

- IaaS examples: Amazon EC2, Microsoft Azure Virtual Machines and Google Compute Engine.
- PaaS examples: Azure App Service, Google App Engine and managed Kubernetes/application platforms such as Amazon EKS with supporting managed services.
- SaaS examples: Microsoft 365, Google Workspace and institutionally hosted SaaS learning-management products.

These are examples, not a provider endorsement. The final procurement decision should compare regional availability, education-sector terms, security certifications, data location, support, integration and total cost.

TASK 7 – EXAMINE XAAS

7.1 Meaning of XaaS

XaaS means Anything-as-a-Service or Everything-as-a-Service. It describes the delivery of technology capabilities as services rather than requiring the university to own and operate every component. IaaS, PaaS and SaaS are common categories; more specialised offerings address data, identity, security, backup, integration and application functions.

7.2 XaaS variants

Variant	Meaning	Possible university use
DBaaS – Database as a Service	Managed database engine, patching, backups and often automated replication.	SIS, library and payment metadata databases, with high availability and point-in-time recovery.
DRaaS – Disaster Recovery as a Service	Provider-assisted replication, orchestration and recovery of systems in a secondary region/site.	Recover critical student services after a regional outage or major incident.
FaaS – Function as a Service	Event-triggered functions execute code without managing persistent servers.	Payment callback processing, result publication notifications and scheduled reports.
BaaS / MBaaS – Backend / Mobile Backend as a Service	Managed backend features such as authentication, push notifications, API support and data synchronisation.	Mobile app login, push notifications and selected backend functions.
SECaaS – Security as a Service	Cloud-delivered security capabilities such as threat detection, email security, web filtering or security monitoring.	Central log analysis, endpoint protection, web application firewall and threat detection.
DaaS – Desktop as a Service	Cloud-hosted virtual desktops delivered to end-user devices.	Specialised lab or administrative desktops where centralised configuration is valuable.

7.3 Two recommended XaaS offerings

Recommendation 1 – DBaaS: A managed relational database is valuable for critical structured records because the provider can automate routine patching, backups, monitoring and high-

availability configuration. The university must still design schemas, access control, encryption, retention, query performance and recovery objectives. It should verify that backups are restorable and that database replicas meet the required recovery point and recovery time.

Recommendation 2 – DRaaS: The university currently has no disaster-recovery site. A managed recovery capability can replicate critical data and support recovery runbooks in a separate region. It should be introduced first for SIS, payment records and essential student services. DRaaS is only effective if failover is tested, dependencies are included, DNS and identity work in the recovery region, and staff know who authorises recovery.

7.4 Risks of relying on multiple providers

- Integration complexity: different identity, logging, billing and support systems can create operational fragmentation.
- Data movement and egress cost: transferring large backups or datasets between providers may be expensive and slow.
- Inconsistent security controls: access policies, encryption defaults and audit capabilities may differ.
- Provider dependency: outages or policy changes at one provider can affect critical workflows.
- Skills and contract burden: teams must understand several consoles, service-level agreements, support processes and renewal dates.

Mitigation: establish a cloud governance framework, standardise identity federation and logging, maintain data classification and exit plans, use open formats where practical, document dependencies, and avoid multi-cloud complexity unless it solves a clear resilience, regulatory or procurement requirement.

TASK 8 – DESIGN THE CLOUD SOLUTION ARCHITECTURE

8.1 Proposed architecture

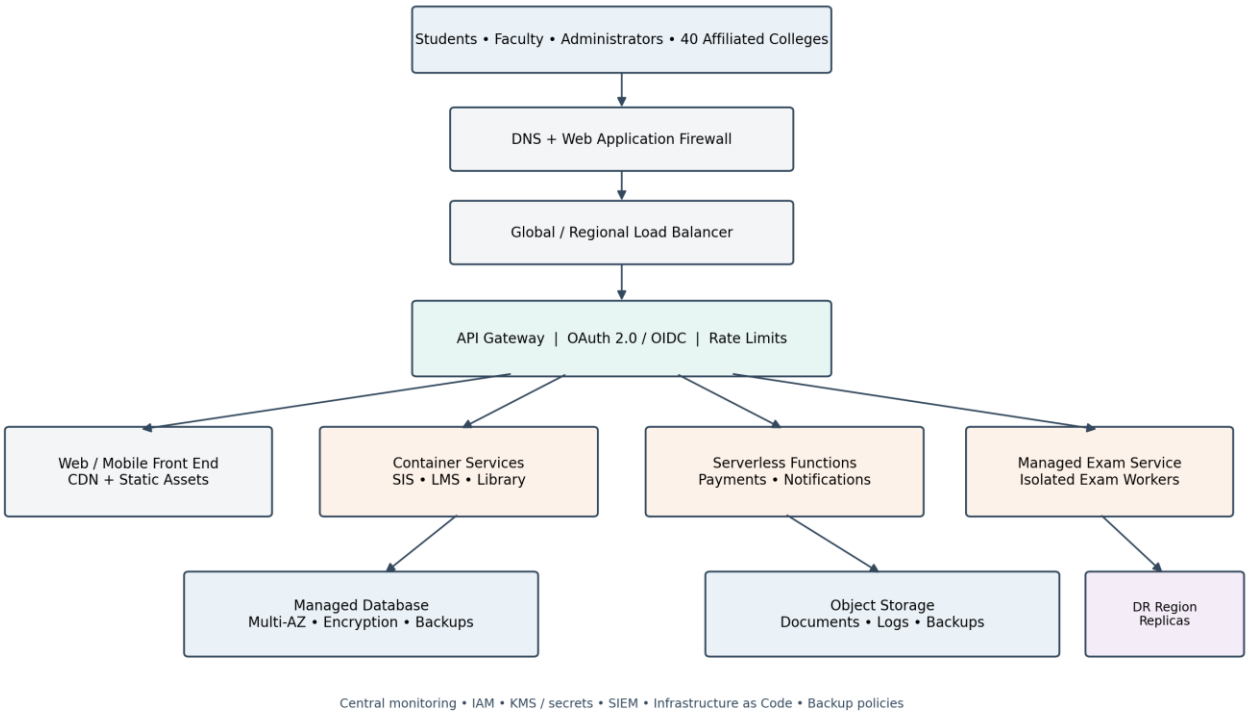


Figure 2. Proposed hybrid-ready cloud architecture for the university. The DR region is a recovery target; actual replication and failover must be configured and tested.

8.2 Component descriptions and service/deployment choices

Component	Proposed model	Purpose and justification
Users and client applications	Public cloud access; web/mobile clients	Students, staff and authorised college users access services using browsers or mobile applications.
DNS, CDN and WAF	Managed cloud service (SaaS/PaaS capability)	Routes requests, caches static assets and filters common web attacks before traffic reaches the application.
Load balancer	Managed service	Distributes traffic across healthy application instances and availability zones.
API gateway	Managed PaaS capability	Provides a controlled entry point for REST APIs, authentication integration, quotas, rate limiting, version routing and request logging.
Web/application tier	Containers on managed orchestration or PaaS	Runs SIS APIs, library services and LMS integrations; stateless replicas can scale horizontally.
Serverless functions	FaaS	Runs event-driven payment callbacks, notifications and scheduled tasks without a continuously running server.
Database tier	DBaaS, private network access	Stores structured student, course, library and payment metadata with encryption, backups and multi-zone availability.
Object storage	Managed object storage	Stores documents, exports, attachments, static content and backup objects with lifecycle policies.
DR region	DRaaS / replicated IaaS and DBaaS	Provides a separate recovery environment with replicated data and documented failover steps.
Identity, keys and monitoring	Managed IAM, KMS/secrets and monitoring services	Centralises identity, privileged access, encryption-key management, logs, alerts and audit evidence.

8.3 Deployment model and network design

The recommended deployment is hybrid during migration and predominantly public cloud for the target architecture. A secure site-to-site VPN or dedicated private connection links the existing campus data centre to the cloud during transition. Application and database tiers should be placed in private subnets; only the necessary edge services should be internet-facing. Separate development, test and production accounts/subscriptions/projects should be used to limit accidental access and blast radius.

8.4 Elasticity for admissions and examination peaks

4. Collect baseline metrics for request rate, CPU, memory, database connections, queue depth, response time and error rate.
5. Define scaling policies using both resource metrics and business signals. For example, add web replicas when CPU or request latency remains above a threshold, and scale background workers when queue depth rises.
6. Keep application services stateless where possible and store session state in a shared managed store or signed token rather than local instance memory.
7. Use a load balancer to distribute traffic across healthy replicas and availability zones.
8. Use caching for public and frequently read information, while ensuring that private student records and fee information are not accidentally cached across users.
9. Set maximum scaling limits, budget alerts and provider quotas. Autoscaling cannot exceed quota limits or compensate for a database bottleneck without database capacity planning.

8.5 Security measures

- Identity and access: central identity provider, SSO, MFA for staff and administrators, least-privilege RBAC, privileged access review and timely deprovisioning.
- Network: private subnets for databases, restricted security groups, WAF, DDoS protection where available, controlled egress and administrative access through approved channels.
- Data: TLS in transit, encryption at rest, managed key rotation, secrets manager, data classification and retention policies.
- Application: secure coding, dependency scanning, patching, input validation, parameterised queries, API rate limits and regular penetration testing.
- Monitoring: central audit logs, immutable security events where supported, alerting for suspicious access and documented incident response.
- Multi-tenancy: each affiliated college receives a tenant identity and scoped permissions. Every data query must enforce tenant context on the server side; tenant ID must never be trusted solely because a client supplied it.

8.6 Availability and disaster recovery

Deploy critical services across at least two availability zones in the primary region, use health checks and redundant load balancing, and configure database high availability. Backups should be encrypted, retained according to policy and copied to a separate failure domain. A recovery region should contain the minimum infrastructure needed to restore priority services.

Define recovery objectives with university leadership. A reasonable initial proposal is RPO (maximum acceptable data loss) of 15 minutes for critical SIS/payment data where replication supports it, and RTO (target time to restore service) of four hours for priority services. These are proposed targets, not guaranteed outcomes; they require provider capability, cost approval and regular recovery exercises.

Availability must be calculated end-to-end. A highly available application tier cannot compensate for a single-zone database, broken identity service, failed DNS, or unavailable campus network. The architecture should include dependency mapping and regular failover drills.

TASK 9 – HANDS-ON IMPLEMENTATION AND DOCUMENTATION

9.1 Proof-of-concept objective and evidence status

This section defines a small reproducible proof of concept: a Flask REST API for a university result lookup, packaged in a Docker container. It demonstrates application packaging and an API endpoint. A local Docker run is a valid containerisation demonstration; a managed PaaS deployment is described as an optional extension.

Important evidence note: the assignment source marks actual execution evidence as partly present/pending. The code and commands below are proposed reproducible instructions with expected outputs. They are not actual execution logs or screenshots. The student must run them and insert genuine screenshots of the terminal, running container and API response before submission.

9.2 Example project structure

```
university-api/\n  app.py\n  requirements.txt\n  Dockerfile
```

9.3 Flask REST API – app.py

```
from flask import Flask, jsonify

app = Flask(__name__)

RESULTS = {
   "STU10245": {
      "studentId": "STU10245",
      "semester": "2026-odd",
      "results": [
         {"courseCode": "CSA15", "grade": "A", "status": "PUBLISHED"},
         {"courseCode": "CSB12", "grade": "B+", "status": "PUBLISHED"}
      ]
   }
}

@app.get("/health")
def health():
   return jsonify({"status": "UP"}), 200

@app.get("/api/v1/students/<student_id>/results")
def get_results(student_id):
```

```
    record = RESULTS.get(student_id)
    if record is None:
        return jsonify({
            "error": "NOT_FOUND",
            "message": "No result record found for the supplied student ID"
        }), 404
    return jsonify(record), 200


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

9.4 requirements.txt and Dockerfile

```
Flask==3.1.0
```

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

For a production deployment, pin dependencies to versions tested by the university, run the process as a non-root user, use a production WSGI server such as Gunicorn, add structured logging, configure secrets outside the image, and avoid embedding real student records in source code.

9.5 Build and run using Docker

```
# Move into the project directory
cd university-api


# Build the container image
docker build -t university-results-api:1.0 .


# Run the container
docker run --name university-api -p 5000:5000 -d university-results-api:1.0


# Confirm that the container is running
docker ps


# View application logs
docker logs university-api
```

9.6 Test the API

```
# Successful lookup – expected HTTP 200
curl -i http://localhost:5000/api/v1/students/STU10245/results


# Missing student – expected HTTP 404
curl -i http://localhost:5000/api/v1/students/UNKNOWN/results


# Health check – expected HTTP 200
curl -i http://localhost:5000/health
```

9.7 Expected response examples (not actual execution evidence)

```
HTTP/1.1 200 OK
Content-Type: application/json

{
 "studentId": "STU10245",
 "semester": "2026-odd",
 "results": [
   {"courseCode": "CSA15", "grade": "A", "status": "PUBLISHED"},
   {"courseCode": "CSB12", "grade": "B+", "status": "PUBLISHED"}
 ]
}
```

```
HTTP/1.1 404 NOT FOUND
Content-Type: application/json

{
 "error": "NOT_FOUND",
 "message": "No result record found for the supplied student ID"
}
```

9.8 PaaS extension procedure

To demonstrate a PaaS component, create a managed web-app service in the selected cloud provider, choose a supported Python runtime, connect the source repository or upload the application package, configure the startup command, set environment variables, enable HTTPS and deploy. Configure a managed database only after defining the schema and access rules. The exact portal steps and free-tier limits vary by provider and region, so the student must record the provider, region, runtime, plan, deployment timestamp and service URL used.

9.9 Screenshot checklist for final submission

Screenshot	What it must visibly prove	Status to complete
Figure 3 – Docker build	Terminal showing successful docker build and image tag.	Insert authentic screenshot after running.
Figure 4 – Running container	docker ps showing university-api and port mapping.	Insert authentic screenshot after running.
Figure 5 – HTTP 200 test	curl/Postman/browser showing successful JSON result.	Insert authentic screenshot after running.
Figure 6 – HTTP 404 test	Unknown student request returning HTTP 404 and error JSON.	Insert authentic screenshot after running.
Figure 7 – PaaS deployment	Cloud console showing deployed app, region and running state; hide secrets.	Insert authentic screenshot if PaaS extension is completed.

After testing, stop and remove the local container with docker stop university-api and docker rm university-api. Keep the source code, Dockerfile, terminal logs and screenshots together so the demonstration can be reproduced. Never include real student personal data, access tokens, passwords or payment details in screenshots.

TASK 10 – EVALUATE AND RECOMMEND THE FINAL SOLUTION

10.1 Three-way comparison

Evaluation area	Continue on-premises	IaaS-based cloud	PaaS/SaaS-oriented cloud
Cost structure	High upfront CapEx for servers, storage, networking, power and facilities; staffing and maintenance continue.	Lower initial CapEx and usage-based compute/storage/network charges; university still pays for OS, middleware and operations.	More OpEx/subscription or managed-service charges; lower infrastructure administration, but usage and per-user costs require control.
Scalability	Limited by purchased hardware and procurement lead time.	Can provision VMs faster, but customer must design scaling and manage OS/application layers.	Often provides managed autoscaling and service capacity; still subject to quotas, product limits and database bottlenecks.
Availability	Depends on local redundancy; current scenario has no DR site.	Can use multiple zones and recovery region, but customer must configure and test them.	Managed services can simplify high availability, but end-to-end availability remains a shared design responsibility.
Security	Direct physical control, but university owns patching, monitoring, physical security and recovery.	Provider secures physical infrastructure; university secures guest OS, applications, identities and data.	Provider manages more of the stack; university still manages identities, data governance, configurations and access.
Operational effort	High hardware, network, OS, backup and facilities workload.	Medium-to-high due to VM, OS and middleware management.	Lower infrastructure effort; requires SaaS/PaaS governance, integration and vendor management.
Lock-in	Lower provider lock-in, but tied to local hardware and legacy systems.	Moderate; VM images are portable but cloud networking and managed services may not be.	Potentially higher where data formats, workflows and proprietary APIs are difficult to export.

10.2 Three-year cost estimate – assumptions

The following is a comparative planning model in Indian rupees (INR). It is intentionally illustrative because the assignment does not provide server purchase prices, cloud region, vendor quotations, licensing terms, data volume, bandwidth, staffing rates or exact workload sizing. Actual procurement must use a provider calculator and written quotations.

- Current environment: twelve physical servers; assumed replacement/refresh and supporting storage/network cost of ₹60 lakh over three years.
- On-premises operations: assumed ₹18 lakh per year for power, cooling, maintenance, backup and support, plus ₹24 lakh per year equivalent staff/operations cost.
- IaaS cloud: assumed ₹12 lakh per year compute/storage/network baseline, ₹8 lakh per year backup/security/monitoring, and ₹18 lakh per year cloud operations/staff effort; initial migration cost ₹10 lakh.
- PaaS/SaaS-oriented: assumed ₹18 lakh per year managed application/database/platform and SaaS subscriptions, ₹5 lakh per year integration/security/backup add-ons, and ₹12 lakh per year operations/governance; initial migration and integration cost ₹14 lakh.

All figures exclude taxes, major application redevelopment, extraordinary data egress, education discounts and large unplanned peak surcharges. The assumptions should be replaced with measured workload and provider quotes before a real decision.

Cost component	On-premises (3 years)	IaaS (3 years)	PaaS/SaaS (3 years)
Hardware / refresh / initial migration	₹60 lakh	₹10 lakh migration	₹14 lakh migration
Infrastructure / platform / subscriptions	Included in refresh; not separately counted	₹12 lakh × 3 = ₹36 lakh	₹18 lakh × 3 = ₹54 lakh

Backup, security, monitoring / support	₹18 lakh × 3 = ₹54 lakh	₹8 lakh × 3 = ₹24 lakh	₹5 lakh × 3 = ₹15 lakh
Staff / operations / governance	₹24 lakh × 3 = ₹72 lakh	₹18 lakh × 3 = ₹54 lakh	₹12 lakh × 3 = ₹36 lakh
Estimated 3-year total	₹186 lakh (₹1.86 crore)	₹124 lakh (₹1.24 crore)	₹119 lakh (₹1.19 crore)

Calculation check: On-premises = 60 + (18 × 3) + (24 × 3) = ₹186 lakh. IaaS = 10 + (12 × 3) + (8 × 3) + (18 × 3) = ₹124 lakh. PaaS/SaaS = 14 + (18 × 3) + (5 × 3) + (12 × 3) = ₹119 lakh.

Interpretation: under these assumptions, PaaS/SaaS is marginally lowest over three years, followed closely by IaaS. The difference is not large enough to decide on cost alone. The university must include functional fit, data protection, exit costs, availability, integration and staff capability. A managed service can be more expensive than expected if per-user, storage, API or premium-support charges grow.

10.3 Sensitivity and cost governance

A 10× peak does not necessarily mean 10× monthly cost if the peak is brief and elastic scaling is effective. However, it can increase compute hours, database capacity, bandwidth, log ingestion and payment/API charges. If the 10× load becomes sustained rather than seasonal, the annual cost may rise materially. Use budget alerts, per-application tags, reserved/committed capacity for predictable baseline demand, autoscaling for bursts, storage lifecycle policies and regular rightsizing. Avoid over-provisioning all workloads for the highest theoretical peak.

10.4 Main risks and mitigations

Risk	Potential impact	Mitigation
Vendor lock-in	High migration cost or inability to move data/workflows.	Use documented APIs, exportable data formats, database backups, container images where practical, contract exit clauses and periodic portability tests.
Data privacy / breach	Exposure of student, staff or payment-related information.	Data classification, encryption, least privilege, MFA, audit logs, secure development, privacy review and contractual controls.
Provider or network downtime	Students may be unable to access results, exams or payments.	Multi-zone design, tested DR, resilient campus connectivity, status communications, offline procedures for critical exams and clear RTO/RPO.
Skills gap	Poor configuration, insecure operations and dependence on consultants.	Training, runbooks, infrastructure as code, peer review, managed services and gradual transfer of knowledge.
Cost overrun	Unexpected consumption, data egress or premium service charges.	Budgets, alerts, tagging, cost allocation, quotas, autoscaling limits and monthly FinOps reviews.
Migration failure / data inconsistency	Duplicate, missing or corrupted student records.	Data profiling, cleansing, reconciliation, staged cutover, read-only freeze windows, rollback plan and business-owner sign-off.

10.5 Phased migration roadmap

Phase	Activities	Why / exit condition
0 – Discover and prepare	Inventory applications, dependencies, data classifications, licences, peak metrics, network links and owners. Establish cloud landing zone, IAM, logging, budgets and policies.	No migration until critical dependencies, security requirements and ownership are known.
1 – Foundation and low-risk services	Move development/test, static public content, monitoring and selected backup copies. Establish VPN/private connectivity and CI/CD.	Validates network, identity, operations and cost controls with limited business risk.

2 – Email and collaboration	Migrate email to a suitable SaaS product after identity, retention, domain and user testing.	Reduces local infrastructure burden and provides a familiar, well-supported service.
3 – Library and LMS integration	Move suitable library discovery APIs and LMS integration components to PaaS/containers; retain legacy components if needed.	Moderate complexity and useful opportunity to test API integration and autoscaling.
4 – SIS and fee-related services	Migrate SIS to IaaS initially or modernise service-by-service; move payment integration behind a secure API gateway.	Highest data and business criticality; proceed only after reconciliation, security and rollback tests.
5 – Exam portal	Deploy isolated, autoscaled application capacity; perform load, failover, security and exam simulation tests.	Exam workload is bursty and high impact; release only after peak testing and operational rehearsal.
6 – Optimise and decommission	Review performance and cost, remove unused servers, validate backups and recovery, and close legacy infrastructure only after sign-off.	Avoid paying for duplicate systems longer than necessary while preserving rollback until stable.

10.6 Final recommendation

Adopt a hybrid transition with a public-cloud target architecture, using IaaS for legacy systems that cannot be changed immediately, PaaS/containers for new and modernised application services, SaaS for email and a suitable LMS, and DBaaS/DRaaS for managed data protection and recovery. This combination balances control, speed, operational effort and cost. It is more appropriate than a full IaaS lift-and-shift because it avoids reproducing all legacy maintenance in the cloud, and more realistic than attempting to replace every application with SaaS at once.

TASK 11 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN

11.1 Impact analysis

The changed requirement materially expands the original problem. The university is no longer serving only its own 25,000 students and 2,000 staff across four campuses; it must also support a mobile app and expose student-record and fee-payment services to 40 affiliated colleges. The expected load is now ten times the current peak, availability must reach 99.9%, and the operating budget remains limited.

- Load assumption changes: the original design was based on seasonal peaks; the revised design must handle a tenfold peak and potentially more concurrent API clients.
- User and trust boundaries change: affiliated colleges are external organisations and must be treated as separate tenants with their own administrators, users, roles and quotas.
- API criticality increases: mobile and partner systems need stable, documented, authenticated APIs. Direct database access or screen-scraping is unacceptable.
- Payment risk increases: payment initiation and status updates must be reliable, idempotent, auditable and protected from replay or duplicate submission.
- Availability expectation becomes explicit: 99.9% is an end-to-end target, not simply a cloud provider's infrastructure claim.

11.2 Availability target calculation

A 99.9% availability target allows 0.1% unavailability. Using a 365-day year:

```
Total minutes per year = 365 × 24 × 60 = 525,600 minutes\nAllowed downtime = 525,600 × 0.001 = 525.6 minutes\n
= 8.76 hours per year
```

The target is demanding but achievable only with an engineered and monitored service. Planned maintenance, provider incidents, application defects, DNS, identity, campus connectivity and partner network failures must be considered in the service measurement definition.

11.3 Evaluation of the original design

The original architecture's load balancer, container services and autoscaling are a good foundation, but a simple single-region deployment is not sufficient for a credible 99.9% end-to-end target if it contains a regional single point of failure. The original virtualization consolidation estimate is also not a 10× capacity plan; it is an average CPU-based estimate for the existing on-premises servers.

Original design element	10× / 99.9% assessment	Required response
Three-host theoretical consolidation	Not a safe capacity plan for a tenfold peak; based only on average CPU and no memory/I/O details.	Use cloud autoscaling and measured load tests; retain N+1 or equivalent resilience for any remaining on-premises cluster.
Single-region application deployment	Can tolerate some component failures if multi-zone, but not a full regional outage.	Use multi-zone primary deployment and a separate recovery region; define active-passive failover for critical services.
REST APIs	Suitable foundation for mobile and partner access, but basic endpoints alone are insufficient.	Add API gateway, versioning, OAuth/OIDC, quotas, tenant-aware authorization, rate limits and audit logs.
Managed database	Useful, but database capacity and connection limits may bottleneck 10× traffic.	Use tested database sizing, connection pooling, read replicas where suitable, caching and queue-based writes where appropriate.
Baseline cost assumptions	Original cost model does not include 10× peak traffic, external tenants or stronger DR.	Recalculate with measured load profiles, traffic estimates, egress, database I/O, support and recovery-region standby costs.
Single university identity boundary	Insufficient for 40 external colleges.	Federated identity or controlled tenant accounts, per-tenant roles, data isolation and tenant-specific audit.

11.4 Decision: MODIFY the original design

Decision: MODIFY. The architecture's core cloud approach should be retained because virtualization, APIs, managed databases and elasticity remain appropriate. However, the service edge, tenant security, availability design, data tier and operational controls must be strengthened. A complete redesign would add unnecessary migration cost and delay; retaining the original design unchanged would fail to address the new partner, load and availability requirements.

11.5 Revised architecture and flow

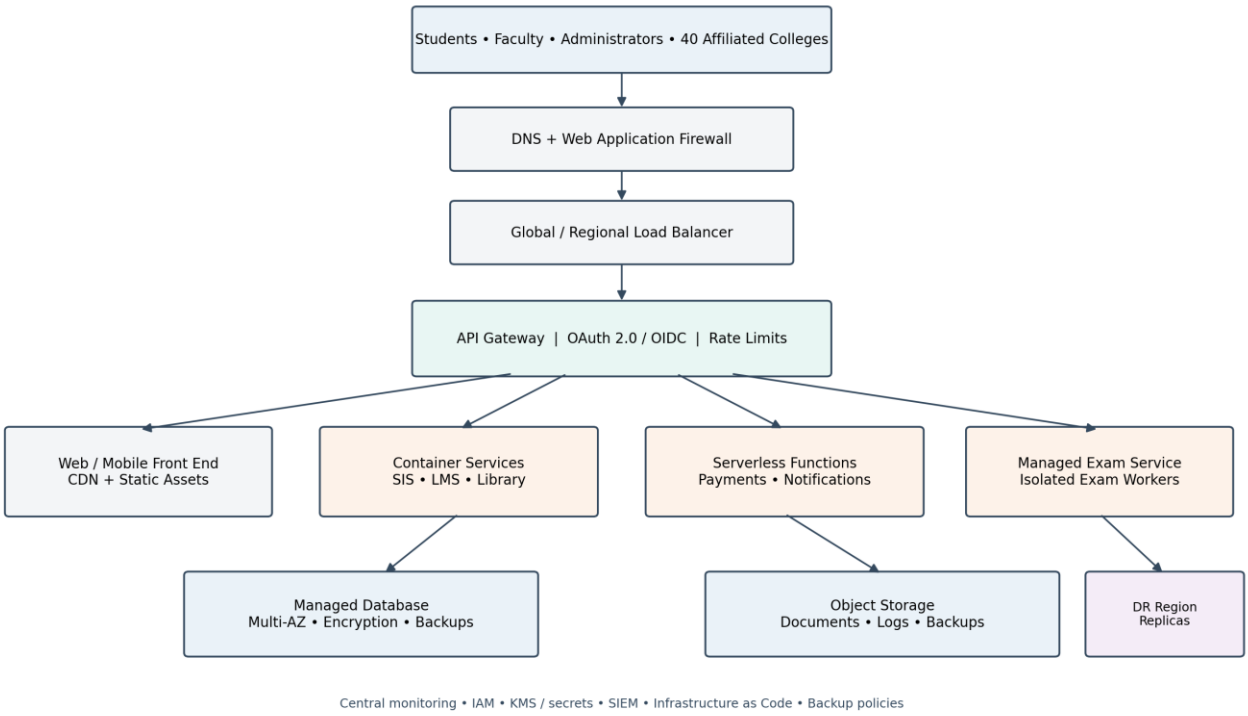


Figure 3. Revised architecture principles: API gateway, multi-zone application services, managed data services and a separate DR region. Tenant-aware policy enforcement applies across every service.

The revised request flow is: mobile app or college client → DNS/CDN/WAF → regional load balancer → API gateway → authentication and tenant policy → application service → database/payment provider. The API gateway validates tokens, client identity, tenant context, quotas and request schemas. Application services enforce authorisation again before returning records or initiating transactions.

11.6 Capacity and scaling strategy

A tenfold peak should be treated as a workload test requirement, not as a simple instruction to multiply every server size by ten. The university must establish the current peak request rate and concurrency, then define a target of ten times that measured peak. For example, if the measured peak is P requests per second, the test target is 10P requests per second. Since the assignment does not provide P, a numeric instance count cannot be calculated honestly.

- Scale stateless web and API containers horizontally across at least two availability zones.
- Use separate worker pools for result publication, notifications and payment events so background work does not consume all interactive capacity.
- Use queues to absorb short bursts and apply back-pressure. Return a clear accepted/pending response for asynchronous tasks.
- Cache public course catalogues and other non-sensitive reference data; do not share personalised result or fee data across tenants.

- Use database connection pooling, query optimisation, indexes and read replicas for read-heavy endpoints. Protect writes and payment transactions with appropriate consistency and idempotency.
- Set per-tenant quotas and fair-use limits so one college cannot consume all shared capacity.
- Run load tests at 1×, 2×, 5× and 10× peak; measure p95/p99 latency, error rate, saturation, queue delay and database performance.

11.7 Multi-tenant security for 40 colleges

A tenant is an affiliated college or the university's own administrative domain. Each tenant must have a unique tenant identifier, tenant-specific user memberships, role assignments and access policies. The API must derive or validate tenant context from a trusted identity claim and server-side membership record. It must not trust an arbitrary tenant ID sent in a request body or URL.

- Authentication: use federation where feasible, or issue managed accounts for college users. Enforce MFA for college administrators and privileged roles.
- Authorisation: apply least-privilege roles such as CollegeAdmin, CollegeStaff, Student and UniversityRegistrar. Restrict each role to permitted operations and records.
- Data isolation: use tenant-scoped database queries and, for highly sensitive boundaries, consider separate schemas or databases. Automated tests must attempt cross-tenant access and confirm denial.
- API protection: assign client credentials and quotas per college, rotate secrets, use signed tokens, validate scopes and monitor abnormal access.
- Audit: record who accessed which student record, for which tenant, at what time and through which client. Protect logs from unauthorised modification.
- Payment controls: separate payment initiation from confirmation, verify signed callbacks, use idempotency keys, reconcile settlements and never expose payment secrets to mobile clients.

11.8 Cost comparison: original versus modified design

The original PaaS/SaaS planning estimate was ₹119 lakh over three years under baseline assumptions. The modified design adds API gateway and WAF capacity, higher application/database consumption during peaks, tenant identity and audit controls, stronger load testing and a standby recovery region. Because request rates, data volume and provider prices are not supplied, the following is a planning allowance rather than a quote.

Cost category	Original 3-year estimate	Modified 3-year planning estimate	Reason for change
Managed app/platform and SaaS	₹54 lakh	₹66 lakh	Higher peak capacity, API gateway and additional service usage.
Backup, security, monitoring	₹15 lakh	₹24 lakh	WAF, tenant audit, enhanced monitoring, recovery replication and testing.
Operations / governance	₹36 lakh	₹42 lakh	Partner onboarding, tenant support, load testing and DR exercises.
Migration / integration	₹14 lakh	₹20 lakh	Mobile API integration, college onboarding, identity and payment hardening.
Total	₹119 lakh	₹152 lakh	Illustrative uplift of ₹33 lakh over three years.

The modified estimate is ₹152 lakh, an increase of ₹33 lakh or approximately 27.7% over the original ₹119 lakh estimate. It is not a claim that 10× load costs only 27.7% more; it assumes that the higher load is a short-lived peak handled by elastic services and that the baseline is still the dominant usage. If 10× becomes sustained, the model must be recalculated and may be substantially higher.

11.9 Trade-offs

The revised architecture improves availability, security and scalability but increases design complexity, observability requirements, cloud consumption and migration effort. A multi-region active-active architecture would provide faster regional continuity but is more expensive and harder to operate, particularly for transactional student and payment data. For a limited budget, multi-zone primary deployment with a warm or pilot-light recovery region is a more proportionate starting point. Recovery time will be longer than active-active and must be clearly documented.

11.10 Evidence-based defence and final position

The decision to modify is supported by four points. First, a tenfold peak cannot be validated using the original average CPU consolidation calculation; it requires measured throughput and load tests. Second, 99.9% allows only 8.76 hours of downtime per year, so a single failure domain and untested recovery process are not acceptable. Third, 40 colleges introduce tenant isolation and partner identity requirements that were not present in the original design. Fourth, a limited budget favours targeted improvements—API gateway, multi-zone deployment, managed database resilience, queues, tenant controls and warm DR—rather than an expensive full active-active multi-cloud redesign.

The final design should be accepted only after a 10× load test meets agreed latency and error thresholds, cross-tenant security tests pass, payment reconciliation is verified, multi-zone failure is exercised, and a recovery drill demonstrates the approved RTO/RPO. Until those tests are completed, 99.9% should be described as a design target, not a proven service result.

OVERALL CONCLUSION

Cloud migration provides the university with a practical path away from fixed, underutilised physical servers and monolithic applications. Virtualization and resource pooling improve utilisation; cloud elasticity addresses seasonal peaks; web services and APIs make university capabilities reusable by mobile applications and partner colleges; and IaaS, PaaS, SaaS and XaaS allow each workload to use an appropriate operating model.

The recommended approach is phased and hybrid during transition, with increasing use of managed platforms and SaaS where they fit. The SIS and other high-value systems should move only after dependency discovery, data reconciliation, security review and rollback planning. Email and suitable LMS capabilities can be evaluated for SaaS, while exam and partner-facing APIs should be designed for burst scaling and strict isolation.

The Day-4 change requires modifying the original design. A tenant-aware API gateway, multi-zone services, resilient database design, queue-based processing, cost controls and a separate recovery region are necessary foundations. The architecture is a proposal until implementation evidence, load

tests, security tests and disaster-recovery exercises demonstrate that it meets the university's requirements.

REFERENCES

10. National Institute of Standards and Technology (NIST). The NIST Definition of Cloud Computing, Special Publication 800-145. https://doi.org/10.6028/NIST.SP.800-145
11. National Institute of Standards and Technology (NIST). Security and Privacy Controls for Information Systems and Organizations, SP 800-53 Rev. 5. https://doi.org/10.6028/NIST.SP.800-53r5
12. National Institute of Standards and Technology (NIST). Zero Trust Architecture, SP 800-207. https://doi.org/10.6028/NIST.SP.800-207
13. Amazon Web Services. AWS Well-Architected Framework. https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html
14. Microsoft. Azure Well-Architected Framework. https://learn.microsoft.com/azure/well-architected/
15. Google Cloud. Architecture Framework. https://cloud.google.com/architecture/framework
16. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.
17. World Wide Web Consortium (W3C). Web Services Description Language (WSDL) 1.1. https://www.w3.org/TR/wsdl
18. OpenAPI Initiative. OpenAPI Specification. https://spec.openapis.org/oas/latest.html
19. Docker Documentation. Docker overview and container documentation. https://docs.docker.com/get-started/overview/
20. Flask Documentation. Quickstart and application deployment guidance. https://flask.palletsprojects.com/
21. OWASP Foundation. OWASP API Security Top 10. https://owasp.org/www-project-api-security/
22. Cloud Security Alliance. Security Guidance for Critical Areas of Focus in Cloud Computing. https://cloudsecurityalliance.org/research/guidance

APPENDIX A – FINAL SUBMISSION COMPLETION CHECKLIST

Item	Before submission
Student details	Fill name, register number, department, semester, faculty and submission date on the cover.
Task 9 execution	Run Docker build/run commands and save actual terminal logs.
Task 9 screenshots	Insert authentic screenshots for build, running container, HTTP 200, HTTP 404 and PaaS deployment if completed.
Cost assumptions	Replace illustrative estimates with actual provider calculator results or clearly retain them as academic assumptions.
Architecture validation	Confirm actual region, availability-zone support, network design, database limits, quotas and recovery options.
Security review	Check access control, encryption, tenant isolation, API scopes, payment security and data retention.
Formatting	Review page breaks, table readability, figure captions and any institution-specific formatting requirements.